

代码风格规范

版本	时间	变更内容
BETA	2024.4.5	初次发布测试版
BETA 0.1.0	2024.10.1 2	
BETA 0.1.1	2024.10.2 7	概括代码风格相关内容
1.0.0	2024.11.2 4	添加第三节“注释”部分、简化第二节“格式”部分内容
1.0.1	2025.11.2 7	添加C++头文件用.hpp后缀的要求，以此区分C和C++的头文件

C++ 风格指南 - 内容目录 — Google 开源项目风格指南

本文是依据Google style guide提炼出的非常简要的概括性内容，如果工作时遇到任何本文没有覆盖到的细则，参考原文即可。

1. 命名

- 大驼峰命名法： `MyGoodName` 。
- 下划线命名法： `my_good_name` 。

少用缩写，必要的缩写（专有名词或者用的非常广泛）视为一个单词。如Hal、Uart、Pwm、Pnp、Ok。

1.1 文件

- 文件名使用下划线命名法 `my_source_file.cc`
- C源文件/头文件用 `.c/.h` 后缀，C++源文件/头文件用 `.cc/.hpp` 后缀。

1.2 类型

枚举、类、结构体、类型别名(typedef、using)等等“数据类型”的名称使用大驼峰命名法

- `enum MyEnum{ }`

- `class MyClass{}`
- `struct MyStruct{}`
- `typedef std::vector<std::vector<double>> Matrix2d`
- `using Matrix2d = std::vector<std::vector<double>>`

1.3 变量

- 普通变量使用下划线命名法 `int my_variable`
- 类的成员变量使用下划线命名法，**并在变量末尾额外加一个下划线**

代码块

```
1  class MyClass {
2      public:
3          int my_member_;
4  };
```

- 没有成员函数的类（结构体），成员变量不加下划线

代码块

```
1  struct MyStruct {
2      int my_member;
3  };
```

- 常量和枚举成员在大驼峰命名法前加一个k

- `const int kMyConstant`
- `constexpr int kMyConstant`
- `enum MyEnum { kMyOption }`

1.4 函数

- 绝大多数函数使用大驼峰命名法
- 唯一功能是取值或者设值的函数（参考标准库容器的 `size()` `begin()` 等等函数），命名法和它作用的变量保持一致

1.5 命名空间

- 使用下划线命名法 `namespace my_namespace{}`

1.6 宏

少写宏！函数和constexpr常量通常都能解决宏能解决的问题。

2. 格式

2.1 clang-format自动格式化

所有人统一使用clang-format进行自动格式化，使用以下规则：

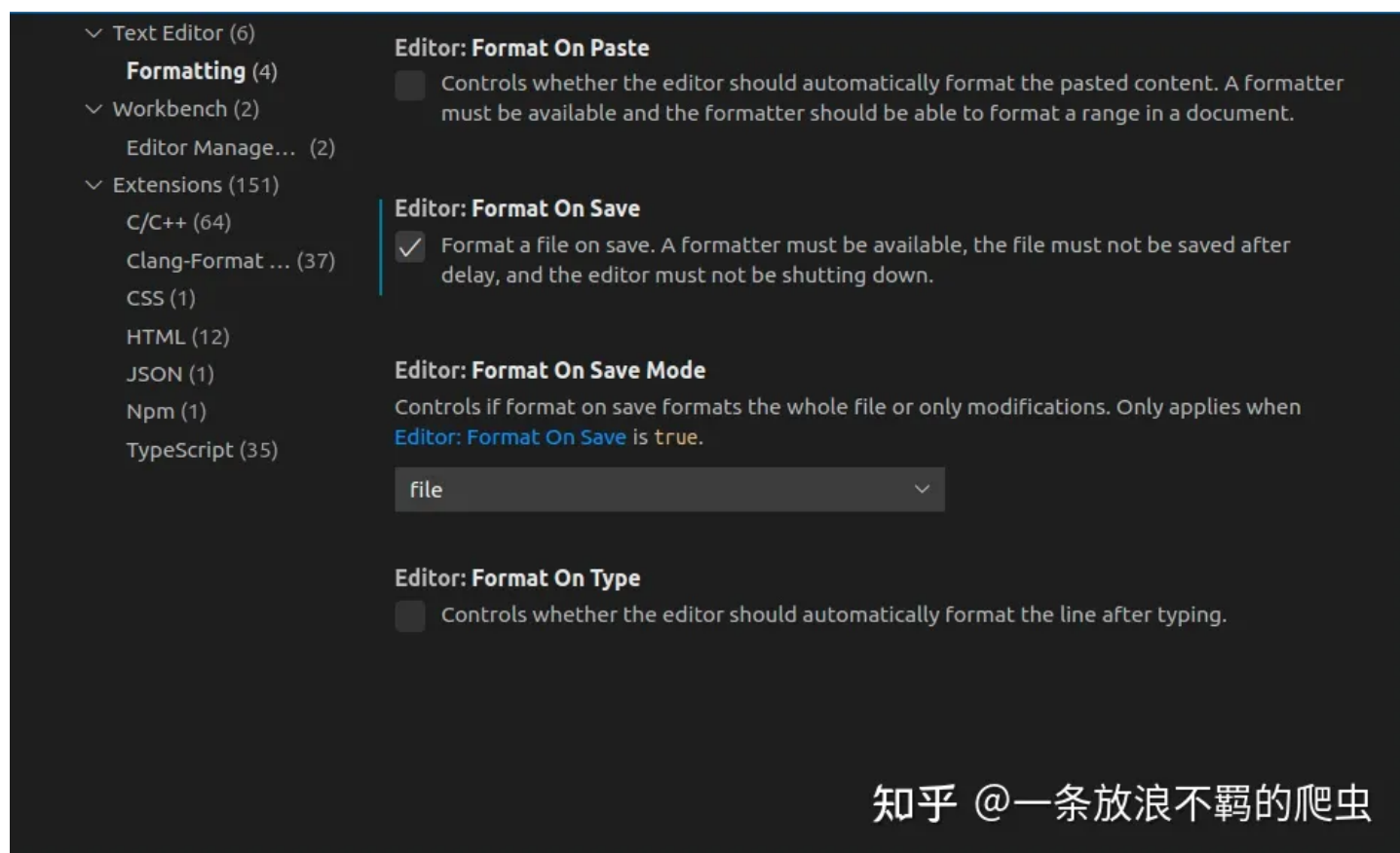
代码块

```
1  Language: Cpp
2  BasedOnStyle: Google
3  ColumnLimit: 120
4  Standard: c++17
5  PointerAlignment: Right
6  SortIncludes: Never
```

在工程根目录下建立一个文件名为 `.clang-format` 的文件，把上述内容粘贴进去即可生效。

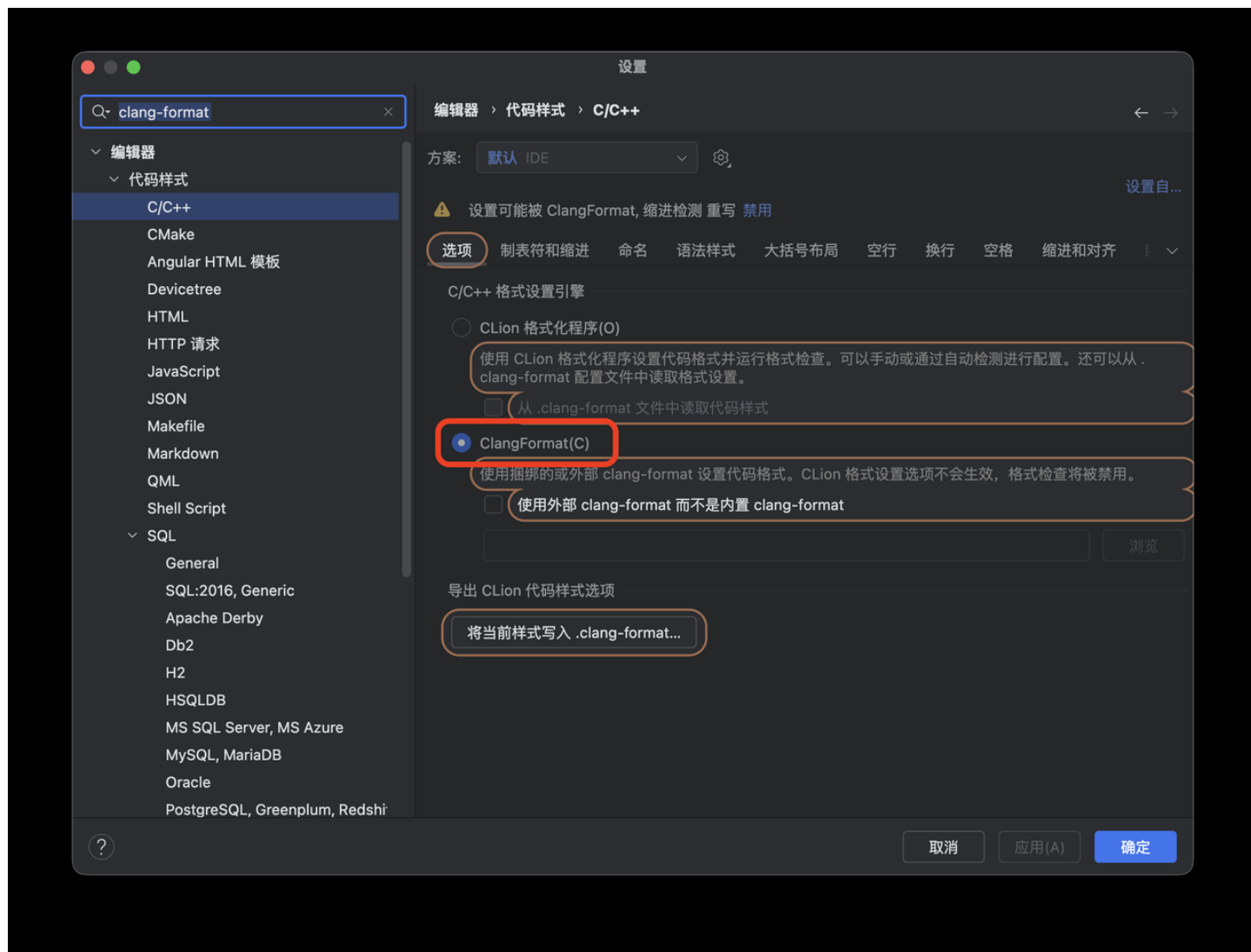
2.1.1 VSCode

1. 安装**C/C++ Extension Pack**插件，按Alt+Shift+F即可格式化当前文件。
2. 保存时自动格式化：打开设置，搜索format，勾选format on save。



2.1.2 Clion

打开设置，搜索 `clang-format`，勾选这一项：



3. 注释

注释采用doxygen格式，举几个常见的例子：

3.1 文件注释

在文件的最开头，写出完整的文件路径、文件名和简介。

代码块

```
1  /**
2   * @file    fsm.hpp
3   * @brief   本文件的内容是底盘和云台两个状态机的定义，使用了一个有限状态机库tinyfsm（先
4   *          去看这个库怎么用）
```

```

5    * @brief    云台状态机的状态有Debug, Normal, NoForce三种, 底盘状态机的状态有Normal,
    FollowGimbal, NoForce三种
6    * @brief    对应整车的3x3种模式组合
7    *
8    * @brief    状态机要对三种事件做出反应: 控制事件Control和底盘、云台各自的模式切换事件
    SwitchMode
9    * @brief    接收到Control事件后, 两个状态机会分别执行一次控制循环, 所以必须持续在主线程
    里dispatch Control事件
10   * @brief    接收到SwitchMode事件后, 对应的状态机会切换到指定的状态
11   */

```

3.2 类注释和函数注释

在类的上面写出这个类的作用。

代码块

```

1    /**
2    * @brief CAN设备的基类
3    * @note 这个类是抽象基类, 用来被CAN设备类继承, 不能实例化。
4    * @note 使用以下几个步骤, 实现一个使用CAN总线通信的设备类:
5    * @note 1. public继承CanDevice
6    * @note 2. 代理构造, 表明自己属于哪一条CAN总线、自己要接收哪些ID的报文
7    * @note 3. 实现RxCallback(const hal::CanMsg *msg)函数, 编写收到报文时的处理逻辑
8    * @note 框架对CAN总线和设备的封装使用观察者模式, CAN设备向CAN总线类"注册"自己, 并且告
    知自己要接收哪些ID的报文;
9    *      基于具体平台实现, CAN总线类会用轮询或接管中断的方式接收所有报文, 每接收到一条报
    文, 它就会寻找有没有注册过想要接收这条报文的设备,
10   *      如果有, 这个设备的RxCallback()函数就会被CAN总线类调用。
11   */
12   class CanDevice {
13       //...
14   };

```

在函数定义或声明前写出这个函数的作用、每个参数的简介和返回值的简介。

代码块

```

1    /**
2    * @brief      calculate crc8
3    * @param[in]   input      data
4    * @param[in]   len        stream length = data + checksum
5    * @param[in]   uc_crc8    init value
6    * @returns     crc8
7    */
8    u8 Crc8(const u8 *input, unsigned int len, u8 uc_crc8) {

```

```

9     while (len--) {
10         const u8 uc_index = uc_crc8 ^ *input++;
11         uc_crc8 = CRC8_TABLE[uc_index];
12     }
13     return uc_crc8;
14 }

```

3.3 变量和枚举成员注释

在变量或枚举成员的同一行后面写一个 `///`，然后在后面开始写只针对这个变量或者成员的注释。

代码块

```

1     enum class Mode {
2         kFollowGimbal, ///  
有力, 无随动
3         kSpin,          ///  
小陀螺
4         kNoForce,        ///  
无力
5     };

```

3.4 补充说明

对代码逻辑的解释在相关位置直接写出来即可：

代码块

```

1     TrajectoryLimiter::TrajectoryLimiter(f32 max_vel, f32 max_accel) :
2         max_vel_(max_vel), max_accel_(max_accel) {
3         ///  
参数验证：确保速度和加速度限制为正数
4         if (max_vel_ <= 0.0f) {
5             max_vel_ = 1e-6f; ///  
设置为极小的正数，防止除零
6         }
7         if (max_accel_ <= 0.0f) {
8             max_accel_ = 1e-6f; ///  
设置为极小的正数，防止除零
9         }
10    }

```